Assignment 3 - Adapter and Bridge Patterns

Design Rationale: Museum Audio Guide System

1. Problem domain

The project models a museum audio guide. A museum may offer different kinds of tours, while the same tour can use different audio playback technologies. In this design, StandardTourGuide and InteractiveTourGuide represent two tour variants. Audio playback is handled through the AudioPlayer interface. The available implementations are LocalAudioPlayer, StreamingAudioPlayer, and LegacyPlayerAdapter.

The important point is that the type of tour and the playback technology are two separate dimensions. A standard tour may use local playback today and streaming tomorrow, while an interactive tour may use the legacy museum hardware without changing the tour class itself.

2. Why Bridge is used

Bridge separates the tour hierarchy from the player hierarchy. AudioGuide is the abstraction and stores only an AudioPlayer reference. StandardTourGuide and InteractiveTourGuide are refined abstractions. AudioPlayer is the implementor interface, while the three player classes are its implementations.

Without Bridge, the project could easily grow into combination classes such as StandardLocalGuide, StandardStreamingGuide, InteractiveLocalGuide, and so on. Every new tour type or player type would multiply the number of combinations. Bridge avoids this by allowing both sides to vary independently.

3. Why Adapter is also required

Bridge alone does not solve the integration of the old museum player. LegacyMuseumPlayer does not implement AudioPlayer and its API is genuinely incompatible with the new interface. The modern contract expects play(AudioTrack) and reports failures through AudioPlaybackException. The legacy class instead accepts separate values such as file path, volume level, and start position, and it reports failures using integer status codes.

LegacyPlayerAdapter wraps LegacyMuseumPlayer and implements AudioPlayer. It converts AudioTrack into the legacy method parameters and translates legacy status codes into meaningful AudioPlaybackException messages. As a result, no legacy-specific constants or failure details reach AudioGuide.

4. Why Adapter alone is not enough

Adapter solves only the compatibility problem. It can make LegacyMuseumPlayer look like an AudioPlayer, but it does not separate the tour hierarchy from the playback hierarchy. Without Bridge, the project would still risk creating separate classes for every combination of tour type and player type. The two patterns solve different parts of the same design problem, which is why they are used together.

5. Required complexity module: Dynamic implementor selection

The selected complexity module is Dynamic implementor selection. AudioPlayerResolver chooses the concrete AudioPlayer at runtime from the input mode. The client does not hard-code one specific player implementation. Provider classes such as LocalAudioPlayerProvider, StreamingAudioPlayerProvider, and LegacyAudioPlayerProvider describe which input they support and create the required player.

This also keeps the resolver open for extension. A new playback implementation can be added with a new provider and registered in the provider list without changing the existing AudioGuide classes or the AudioPlayer interface.

6. Open/Closed Principle

Axis	How the design stays open for extension
New tour type	Create another AudioGuide subclass and reuse any existing AudioPlayer implementation.
New player type	Create another AudioPlayer implementation and a matching provider without changing existing guide classes.

7. Main limitation

The project simulates playback instead of using real audio files, network streaming, or physical museum hardware. This keeps the assignment focused on design patterns, but it means the current implementation does not test real device latency, file availability, or network failures. Those features could be added later behind the existing AudioPlayer interface.

8. Final design summary

Bridge handles two independent dimensions: tour behavior and playback technology. Adapter allows one incompatible legacy player to participate in the AudioPlayer side of that Bridge without modifying its source code. Dynamic runtime selection chooses the concrete player from input. Together, these decisions keep the design small, understandable, and extendable.